IA pour les Jeux
Licence Informatique et Vidéoludisme

ASP

Nicolas Jouandeau

n@up8.edu

2026

des problèmes difficiles : combinatoires, NP, ...

- puzzles crypto-arithmétiques : des lettres, des nombres, des sommes
- puzzles arithmétiques : des nombres, des opérateurs, des résultats
- puzzles
  - carré latin : unicité en ligne et en colonne
  - sudoku : unicité en ligne, en colonne et en région
  - light up : placer des lampes pour éclairer une grille contrainte
  - kakuro : placer des nombres pour satisfaire des sommes croisées
  - morpion solitaire : plus longue séquence posssible sur une grille contrainte
- problème des n reines : placer des reines sans menace sur un échiquier
- problème du cavalier : visiter chaque case d'un échiquier sans repasser par une case
- problème du sac à dos : remplir le plus possible un sac de volume fixé
- problème du voyageur de commerce : chemin de coût mini dans un graphe
- problème k-centre : couverture mini de n villes avec k centres
- problème d'ordonnancement : ordonner des tâches pour un but avec ou sans temps max

programmation logique, contraintes, domaines, SAT et modèles stables

1. la programmation logique (PL 70-75) [cours précédent]
2. la programmation logique avec contraintes et domaines (CLP 80-10) [cours précédent]
   - avec des entiers (CLPFD)
   - avec des booléens (CLPB)
   - avec des rationnels (CLPQ)
   - avec des réels (CLPR)
   - avec d'autres contraintes (CLPCHR)
3. SAT (60) [slides 66-76]
4. SMT : SAT avec domaines (93-99) [cours précédent]
5. ASP : SAT avec contraintes explicites et modèles stables (88-08) [slides 4-44]
6. conclusion/perspectives/bibliographie [slides 45-47]

- résoudre un problème combinatoire
  - définir le problème selon des règles logiques
  - définir la résolution selon une heuristique/politique de parcours

de SAT vers ASP (des connaissances complètes aux connaissances incomplètes)

- en SAT, on a la négation classique
  - manipulation de booléens (*i.e.* connaissances complètes, vraies ou fausses)
  - $\neg a$ est vrai si a est faux
  - le monde est explicitement clos
  - le raisonnement est monotone
  - propagation unitaire dans CDCL
- en ASP, on a la négation par défaut
  - manipulation de bases de connaissances incomplètes
  - `not a` est vrai si a n'est pas prouvé
  - le monde est implicitement clos
  - le raisonnement est non monotone
  - on peut exprimer la négation classique avec `-a`
  - CDNL (1997-2008, variante de CDCL avec identification des *nogoods*)
    - propagation unitaire (obligatoirement une vérité fausse dans un nogood)
    - propagation par règles (conflits de causalité et conflits de justification)
    - propagation par stabilité (conflits de circularité)
    - propagation par ensembles non fondés (conflits de cohérence)
    - propagation par contraintes (conflits de minimalité)

Answer Set Programming (ASP) ?

- définition formelle en 1988 dans "*The Stable Model Semantics for Logic Programming*"
  par Michael Gelfond et Vladimir Lifschitz
  (Texas Tech University)
- interpréter les programmes logiques avec négation comme des théories non monotones permettant plusieurs solutions (modèles, "mondes possibles", ensembles réponses, *answer sets*)
- motivations
  - donner un sens formel à la négation en programmation logique
  - obtenir une résolution indépendante de l'ordre
- conséquences (sachant réduction GL $\Longleftrightarrow$ CDNL)
  - accepter la logique non monotone résultant de l'ordre dans un programme logique
    (*i.e.* ne pas avoir de contrainte entre ordre et résultat)
  - une exécution = génération d'une ou plusieurs solutions (modèles)
  - une solution = application de CDNL (variante de CDCL) sur un programme logique
- bénéfices (+ ou - discutables)
  - intuitivité = affirmer l'absence de solution en présence d'indécision (*i.e.* `a :- not a.`)
  - crédulité formelle = plusieurs réponses possibles

noyau d'ASP

- un programme logique = un ensemble de règles de la forme $c \leftarrow a_1, a_2, ..., a_n, not_b_1, not_b_2, ..., not_b_m$
  - avec $n \geq 0$ et $m \geq 0$
  - a_i et b_i des atomes propositionnels
  - $c = \texttt{head}\ (r)$
  - $\{\ a_1, a_2, ..., a_n\ \} = \texttt{body}^+\ (r)$
  - $\{\ not_b_1, not_b_2, ..., not_b_m\ \} = \texttt{body}^-\ (r)$
- une solution du problème posé est nommée *answer set*
- chaque règle correspond à une contrainte de causalité
  - Si tous les a_i appartiennent à un *answer set* et si aucun des b_j n'y appartient, alors c doit appartenir à cet *answer set*.

modèles stables et réduction GL

- pour un programme nommé P et une interprétation nommée M
- la réduction GL nommée P^M est obtenue par
  - suppression des règles avec `not a`, sachant `a ∈ M`
  - suppression de tous les littéraux négatifs `not a` restants (*i.e.* toutes les autres négations)
- si M est le plus petit modèle de P^M alors M est un modèle stable

pour un programme P

```
a :- not b.
b :- not c.
c :- not b.
```

pour un modèle $M = \{ a, b \}$, on a P^M

```
b.
```

- $\{ b \}$ est le plus petit modèle de P^M
- comme $\{ b \} \neq \{ a, b \}$, M n'est pas stable

pour un modèle $M = \{ b \}$, on a P^M

```
b.
```

- $\{ b \}$ est le plus petit modèle de P^M
- M est stable

pour un modèle $M = \{ a, c \}$, on a P^M

```
a.
c.
```

- $\{ a, c \}$ est le plus petit modèle de P^M
- M est stable

nogoods et ensembles non fondés

- principe (passer des contraintes implicites en SAT aux contraintes explicites en ASP)
  - un nogood est un truc faux
  - un nogood est un ensemble de littéraux dont au moins un est faux
  - pour un ensemble de littéraux $L_1, L_2, ..., L_n$
  - si $L_1, L_2, ..., L_{n-1}$ sont tous vrais, alors L_n est faux
  - un nogood est unitaire si $n-1$ littéraux sont vrais
- les différents types de nogood
  - **causalité** : pour $c = \texttt{body}^+(r) + \texttt{body}^-(r)$, on a un nogood $\{\ a_1, ..., a_n, not_b_1, ..., not_b_m, not_c\ \}$
    (il n'est pas possible que corps soit vrai et tête soit fausse)
  - **justification** : pour un atome a avec des règles $r_1, ...r_n$, on a un nogood
    $\{\ a, not_\texttt{body}(r_1), ..., not_\texttt{body}(r_n)\ \}$
    (chaque atome doit être justifié)
  - **négation** : on a deux nogoods $\{\ not_a, a\ \}$ et $\{\ \neg not_a, \neg a\ \}$
    (une affectation et sa négation n'est pas possible)
  - **non-fondation** : si chaque a_i de U est supporté par des règles dépendants d'élément de U, alors U est non fondé et on a un nogood $\{\ a_1, ..., a_n\ \}$
    (les atomes d'un ensemble non fondé ne peuvent pas être tous vrais)
- un modèle stable n'active aucun nogood
- éviter les nogoods permet de guider la recherche de solutions

on a plusieurs nogood pour chaque règle

- pour $c \leftarrow \texttt{body}\ (r) = \{\ a_1, a_2, ..., a_n, not_b_1, not_b_2, ..., not_b_m\ \}$
- on a les nogoods de corps
  - $\{\ not_a_i\ ,\ \texttt{body}\ (r)\ \}$
  - $\{\ b_j\ ,\ \texttt{body}\ (r)\ \}$
  - $\{\ not_\texttt{body}\ ,\ \texttt{body}\ (r)\ \}$
- on a le nogood de tête
  - $\{\ not_c\ ,\ \texttt{body}\ (r)\ \}$

reprenons le programme P

```
a :- not b.
b :- not c.
c :- not b.
```

il n'est pas possible que corps soit vrai et tête soit fausse (causalité)

- $\{ not_b, not_a \}, \{ not_c, not_b \}, \{ not_b, not_c \}$

chaque atome doit être justifié (justification)

- $\{ a, b \}, \{ b, c \}, \{ c, b \}$

une affectation et sa négation n'est pas possible (négation)

- $\{ not_a, a \}, \{ \neg not_a, \neg a \}, \{ not_b, b \}, \{ \neg not_b, \neg b \}, ...$

pas de génération systématique de tous les nogoods

- "appris" (*i.e.* constatés) lors de l'évaluation d'un modèle candidat

prenons un autre programme P

```
a :- b.
b :- a.
```

les atomes d'un ensemble non fondé ne peuvent pas être tous vrais (non-fondation)

- $\{ a, b \}$
  - a est justifié par b
  - b est justifié par a
  - $\{ a, b \}$ est un modèle non fondé

retour au programme P

```
a :- not b.
b :- not c.
c :- not b.
```

évaluation du modèle $\{\ a, b\ \}$ avec CDNL

- affectation courante $\{\ a = true,\ b = true,\ c = false\ \}$
- causalité ✓
- justification ✗
- on mémorise le nogood $\{\ a, b\ \}$

évaluation du modèle $\{\ b\ \}$ avec CDNL

- affectation courante $\{\ a = false,\ b = true,\ c = false\ \}$
- causalité ✓
- justification ✓
- négation ✓
- non-fondation ✓
- $\{\ b\ \}$ est un modèle stable

toujours avec le programme P

```
a :- not b.
b :- not c.
c :- not b.
```

évaluation du modèle $\{\ a, c\ \}$ avec CDNL

- affectation courante $\{\ a = \mathit{true},\ b = \mathit{false},\ c = \mathit{true}\ \}$
- causalité ✓
- justification ✓
- négation ✓
- non-fondation ✓
- $\{\ a, c\ \}$ est un modèle stable

principe des solveurs ASP comme Clingo (Gringo+Clasp)

- décrire un problème avec ASP dans une syntaxe proche de Prolog
  - faits, règles, contraintes
- instancier pour éliminer les variables (avec le *grounder* Gringo)
  - éliminer les règles inutiles
  - utiliser/insérer des symboles conditionnels
  - réduire les possibilités d'explosion combinatoire
  - identifier les domaines des variables
  - remplacer les variables par toutes les valeurs possibles
- générer un/des modèle/s stable/s (avec le *Solver* Clasp)
  - transformer les règles ASP en contraintes SAT avec négation, choix, agrégation et optimisation
  - appliquer CDNL (variante de CDCL) avec apprentissage et back-jump
    - appliquer la propagation unitaire
    - propager les atomes affectés à une valeur
    - choisir un atome et lui affecter une valeur
    - détecter les conflits (*i.e.* les nogoods)
    - si conflit alors apprendre un nogood et revenir à l'affectation la plus pertinente (back-jump)
    - sinon on a une solution
- interpréter le/les modèle/s en réponse/s

programme avec deux faits et une règle

```
f(a,b).
f(b,c).
g(X,Y) :- f(X,Y).
```

grounding complet de tous les atomes

```
f(a,b). f(b,c).
g(a,a) :- f(a,a). g(b,b) :- f(b,b). g(c,c) :- f(c,c).
g(a,b) :- f(a,b). g(a,c) :- f(a,c). g(b,a) :- f(a,b).
g(b,c) :- f(b,c). g(c,a) :- f(c,a). g(c,b) :- f(c,b).
```

grounding naïf réduit

```
f(a,b). f(b,c).
g(a,b) :- f(a,b).
g(b,c) :- f(b,c).
```

résolution : un seul modèle

```
{ f(a,b) , f(b,c) , g(a,b) , g(b,c) }
```

éviter d'ajouter des faits ou des règles inutiles

- éviter le grounding d'éléments qui compliquent artificiellement le problème pour le solver
- contraindre au maximum la combinatoire des atomes dans le corps des règles avec des atomes fondés

termes, atomes, faits et règles

- un terme = un objet du domaine (ni vrai, ni faux)
- un terme = une constante, une variable, une fonction
- un atome = une assertion logique, avec un prédicat et un ou plusieurs termes
  `p(a,b,...,n)`
- un atome peut être vrai ou faux dans un ensemble réponse (*answer set*)
- un fait déclare la valeur de vérité d'un atome
- une règle est usuellement une règle de causalité
- une règle possède une tête et un corps
  `a(X,Y) :- b(X), c(Y), d(X,Y).`
  - `X` et `Y` sont des variables
  - `a(X,Y)` est la tête
  - `b(X), c(Y), d(X,Y)` est le corps
  - `a` vrai si `b`, `c` et `d` vrais
- les faits et les règles se terminent par des points

atomes en ASP

- atomes simples (affirment un fait) sont notés `p(a,b,...,n)`
- atomes avec négation classique (ou négation forte) sont notés `- p(a,b,...,n)`
- atomes avec négation par défaut (ou négation comme échec) sont notés `not p(a,b,...,n)`
- atomes arithmétiques (pour comparer) sont notés `X < Y` ou `X = Y` ou `X > Y` ou `X != Y`
- atomes d'agrégats (pour compter, sommer) sont notés `#count{...} > N` ou `#sum{...} > N`
- attention :
  - seuls les atomes simples sont des prédicats appliqués à des termes
  - la négation classique déclare un prédicat faux
  - la négation par défaut déclare un prédicat non prouvé vrai
- un atome *ground*
  - est un atome sans variable (*i.e.* avec constantes ou fonctions uniquement)
  - est complètement déterminé
- un atome *non ground*
  - est un atome avec au moins une variable
  - correspond à un ensemble d'atomes *ground*

variables sûres, et *domain* prédicat

- toute variable doit être sûre
  - doit apparaître dans un atome positif dans le corps
  - ne doit pas être uniquement dans une comparaison
- conséquence
  - règle invalide : `a(X) :- not b (X).`
  - règle invalide : `a(X) :- X > Y.`
  - règle valide : `a(X) :- b(X), not c (X).`
  - règle valide : `a(X) :- b(X), c(Y), X > Y.`
- un *domain* prédicat définit le domaine d'une variable
  (*i.e.* définit l'ensemble des valeurs possibles d'une variable, permet de définir une borne de *grounding*)
  - `rows(1..3).` utilise le *domain* prédicat d'arité 1 nommé `rows/1`
  - si `rows(X)`, alors $X \in \{1,2,3\}$
  - on peut également énumérer les valeurs d'un domaine avec `cols(1;2;3).`
  - on peut également énumérer les symbols d'un domaine avec `day(mon;tue;wed;thu;fri).`
- un *domain* prédicat est pur s'il est défini par des faits, toujours positif et jamais en tête de règle

atomes d'arithmétique

```
left    (7).                %% 111
right           (2).        %% 010
plus    ( L +  R ) :- left(L), right(R).
minus   ( L -  R ) :- left(L), right(R).
uminus  (   -  R ) :-          right(R).
times   ( L *  R ) :- left(L), right(R).
divide  ( L /  R ) :- left(L), right(R).
modulo  ( L \  R ) :- left(L), right(R).
absolute(|  -  R|) :-          right(R).
power   ( L ** R ) :- left(L), right(R).
bitand  ( L &  R ) :- left(L), right(R).
bitor   ( L ?  R ) :- left(L), right(R).
bitxor  ( L ^  R ) :- left(L), right(R).
bitneg  (   ~  R ) :-          right(R).
```

- dont le modèle est
  { right(2), left(7), plus(9), minus(5), uminus(−2), times(14), divide(3), modulo(1), absolute(2), power(49), bitand(2), bitor(7), bitxor(5), bitneg(−3) }

atomes de comparaison

```
sym(1).  sym(a).  sym(f(a)).
eq  (X,Y) :- X =  Y, sym(X), sym(Y).
neq(X,Y) :- X != Y, sym(X), sym(Y).
lt  (X,Y) :- X <  Y, sym(X), sym(Y).
leq(X,Y) :- X <= Y, sym(X), sym(Y).
gt  (X,Y) :- X >  Y, sym(X), sym(Y).
geq(X,Y) :- X >= Y, sym(X), sym(Y).
```

- dont le modèle est

{ sym(1), sym(a), sym(f(a)),
eq(1,1), eq(a,a), eq(f(a),f(a)),
neq(a,1), neq(f(a),1), neq(1,a), neq(f(a),a), neq(1,f(a)), neq(a,f(a)),
lt(1,a), lt(1,f(a)), lt(a,f(a)),
leq(1,1), leq(1,a), leq(a,a), leq(1,f(a)), leq(a,f(a)), leq(f(a),f(a)),
gt(a,1), gt(f(a),1), gt(f(a),a),
geq(1,1), geq(a,1), geq(f(a),1), geq(a,a), geq(f(a),a), geq(f(a),f(a)) }

contraintes d'intégrité

- une contrainte est une règle avec une tête vide
  - elle exprime ce qui n'est pas possible
  - elle ne pas être uniquement négative
  - ses variables doivent être sûres (*i.e.* supportées par des faits ou des règles)

exemple

```
a :- not b.
b :- not a.
c.
:- c, not b.
```

- `c` est un fait
- `c` et `not b` ne peuvent pas être simultanéement vrais
- sachant que `c` est vrai, `not b` est proscrit
- les modèles candidats sont { `c, a` } { `c, not a` } { `c, b` } { `c, a, b` } { `c, not a, b` }
- `a` demande d'avoir `not b`, hors `not b` est proscrit
- les modèles candidats deviennent { `c, not a` } { `c, b` }
- ce qui se réduit au seul modèle { `c, b` } avec la 2ème règle

3-coloration d'un graphe à 3 sommets et 2 arêtes

```
node(1..3).
edge(1,2). edge(2,3).
red(X):-node(X), not green(X), not blue(X).
green(X):-node(X), not red(X), not blue(X).
blue(X):-node(X), not green(X), not red(X).
:-edge(X,Y),red(X),red(Y).
:-edge(X,Y),green(X),green(Y).
:-edge(X,Y),blue(X),blue(Y).
```

- les sommets sont colorés
- deux sommets connectés ne peuvent pas être de la même couleur
- on a $\binom{3}{1} \times \binom{2}{1} \times \binom{2}{1} = 12$ modèles possibles
- un modèle est { `n(1)`, `n(2)`, `n(3)`, `edge(1,2)`, `edge(2,3)`, `red(1)`, `blue(2)`, `red(3)` }

avec 2 faits supplémentaires

```
red(1).
blue(2).
```

- les modèles possibles deviennent
  - { `n(1)`, `n(2)`, `n(3)`, `blue(2)`, `red(1)`, `edge(1,2)`, `edge(2,3)`, `green(3)` }
  - { `n(1)`, `n(2)`, `n(3)`, `blue(2)`, `red(1)`, `edge(1,2)`, `edge(2,3)`, `red(3)` }

littéral conditionnel

- permet d'inclure un littéral dans un modèle si une condition est satisfaite
- `b : c.`
  - `b` est vrai si `c` est vrai
- `a :- b : c.`
  - `a` est vrai quand
  - `c` est faux quelque soit `b`
  - `b` et `c` tous les deux vrais
- utilisés dans les agrégats et les règles de choix

règles de choix

- syntaxe générale : `L { a1; a2; ...; an } U.`
  - `L` est le nombre minimum d'atomes vrais
  - `U` est le nombre maximum d'atomes vrais
  - les `ai` sont des atomes
- `{ a }.`
  - `a` est vrai ou faux
  - les *answer sets* sont `{}` et `{ a }`

règles de choix (suite)

- `{ a ; b ; c}.`
  - toutes les combinaisons des éléments de l'ensemble `{a,b,c}` sont vraies
  - les *answer sets* sont `{}`, `{a}`, `{b}`, `{c}`, `{a,b}`, `{a,c }`, `{b,c}` et `{a,b,c}`
- `1 { a ; b ; c}.`
  - au moins un élément de l'ensemble `{a,b,c}` est vrai
  - les *answer sets* sont `{a}`, `{b}`, `{c}`, `{a,b}`, `{a,c }`, `{b,c}` et `{a,b,c}`
- `1 { a ; b ; c} 1.`
  - exactement un élément de l'ensemble `{a,b,c}` est vrai
  - les *answer sets* sont `{a}`, `{b}` et `{c}`
- `{ a ; b ; c} 2.`
  - au plus deux éléments de l'ensemble `{a,b,c}` sont vrais
  - les *answer sets* sont `{a}`, `{b}`, `{c}`, `{a,b}`, `{a,c }` et `{b,c}`

règles de choix (suite)

- `{ p(X) } :- q(X).`
  - pour chaque `X` tel que `q(X)` vrai, il est possible d'avoir `p(X)` vrai
  - les *answer sets* dépendent des `q(X)` vrais

exemple

```
q(1). q(2).
{p(X)} :- q(X).
```

- les *answer sets* sont `{}`, `{p(1)}`, `{p(2)}` et `{p(1),p(2)}`

ne pas confondre règle de choix et règle de causalité

- `{ p } :- q.`
  - si `q` est vrai, il est possible d'avoir `p` vrai
  - les *answer sets* sont `{}`, `{q}` et `{p,q}`
- `p :- q.`
  - si `q` est vrai, alors `p` est vrai
  - les *answer sets* sont `{}` et `{p,q}`

configurations possibles pour un jeu à 2 joueurs sur un plateau à 3 cases

```
position(1..3).
player(x;o).
1{ at(Who,Pos) : position(Pos) }1 :- player(Who).
:- at(x,X), at(o,Y), X=Y.
```

- pour chaque joueur, on a un `at` à une `position`
- le joueur `x` et le joueur `o` ne peuvent pas occuper la même position
- il y a $\binom{3}{2} \times 2 = 6$ possibilités
- les six modèles possibles sont

```
position(1) position(2) position(3) player(x) player(o) at(x,1) at(o,2)
position(1) position(2) position(3) player(x) player(o) at(x,1) at(o,3)
position(1) position(2) position(3) player(x) player(o) at(x,2) at(o,3)
position(1) position(2) position(3) player(x) player(o) at(o,2) at(x,3)
position(1) position(2) position(3) player(x) player(o) at(o,1) at(x,3)
position(1) position(2) position(3) player(x) player(o) at(o,1) at(x,2)
```

les agrégations

```
p(1..9).
cnt(X) :- X = #count { Y : p(Y) }.
sum(X) :- X = #sum { Y : p(Y) }.
sump(X) :- X = #sum+ { Y : p(Y) }.
min(X) :- X = #min { Y : p(Y) }.
max(X) :- X = #max { Y : p(Y) }.
```

- nombre, somme, somme des positifs, minimum et maximum d'un ensemble d'atomes dans un modèle

```
p(1) p(2) p(3) p(4) p(5) p(6) p(7) p(8) p(9) cnt(8) sum(4) min(-3) max(4) sump(10)
```

autre solution pour un jeu à 2 joueurs sur un plateau à 3 cases

```
position(1..3).
player(x;o).
{ at(Who,Pos) } :- position(Pos), player(Who).
:- at(x,X), at(o,Y), X=Y.
:- 1 != #count { Y : at(x,Y) }.
:- 1 != #count { Y : at(o,Y) }.
```

- pour un joueur et une `position`, on peut avoir un `at`
- le joueur `x` et le joueur `o` ne peuvent pas occuper la même position
- le joueur `x` doit avoir une seule `position`
- le joueur `o` doit avoir une seule `position`

énumérer les atomes et les combinaisons d'atomes

```
a(1). a(2).
b(10). b(11). b(12).
cnta(X) :- X = #count { Y : a(Y) }.
cntb(X) :- X = #count { Y : b(Y) }.
cnt1(X) :- cnta(X1), cntb(X2), X=X1+X2.
cnt2(X) :- X = #count { Y,Z : a(Y),b(Z) }.
```

- `cnta` compte les atomes `a`
- `cntb` compte les atomes `b`
- `cnt1` compte les atomes `a` et `b`
- `cnt2` compte les combinaisons d'atomes `a` et `b`

```
a(1) a(2) cnta(2) b(10) b(11) b(12) cntb(3) cnt1(5) cnt2(6)
```

minimiser ou maximiser avec `#minimize` ou `#maximize`

```
a(1..2).
b(10..12).
1 { c(X,Y) : a(X) , b(Y) } 1.
#minimize { Z : a(X), b(Y), Z=X+Y }.
```

- peut aboutir à un unique modèle

```
a(1) a(2) b(10) b(11) b(12) c(1,11)
```

construire des listes dans un modèle

```
#const size = 4.
number(1..3).

1{list(X,1):number(X)}1.
list((E,L), K+1):- number(E), list(L, K), K<size.

full(X):-list(X,size).

cnt(X) :- X = #count { Y : full(Y) }.

#show full/1. #show cnt/1.
```

- on a pas les crochets `[]` et le pipe `|` de la programmation logique de Prolog
- `full` est vrai pour les listes de taille 4
- `cnt` compte les listes de taille 4

```
full((1,(3,(1,1)))) full((2,(3,(1,1)))) full((3,(3,(1,1)))) full((1,(2,(1,1))))
full((2,(2,(1,1)))) full((3,(2,(1,1)))) full((1,(1,(1,1)))) full((2,(1,(1,1))))
full((3,(1,(1,1)))) full((1,(3,(2,1)))) full((2,(3,(2,1)))) full((3,(3,(2,1))))
full((1,(2,(2,1)))) full((2,(2,(2,1)))) full((3,(2,(2,1)))) full((1,(1,(2,1))))
full((2,(1,(2,1)))) full((3,(1,(2,1)))) full((1,(3,(3,1)))) full((2,(3,(3,1))))
full((3,(3,(3,1)))) full((1,(2,(3,1)))) full((2,(2,(3,1)))) full((3,(2,(3,1))))
full((1,(1,(3,1)))) full((2,(1,(3,1)))) full((3,(1,(3,1))))
cnt(27)
```

installer clingo (gringo+clasp)

- `https://potassco.org/doc/start/`
- pour avoir clingo avec les macports sur MacOs
  ```
  $> sudo port install clingo
  ```
- pour avoir les bindings python
  ```
  $> python -m venv clingo-env
  $> source clingo-env/bin/activate
  $> pip install clingo
  ```

utiliser clingo

- obtenir un modèle
  ```
  $> clingo prg.lp
  ```
- obtenir tous les modèles
  ```
  $> clingo prg.lp 0
  ```

installation de clingo (gringo+clasp)

- doc de clingo `https://potassco.org/doc/start/`
- install de clingo avec les macports sur MacOs

```
$> sudo port install clingo
```

- doc des bindings python `https://potassco.org/clingo/python-api/5.4/`
- install des bindings python

```
$> python -m venv clingo-env
$> source clingo-env/bin/activate
$> pip install clingo
```

énumérer des positions sur une grille

```
%%% to show possible queen positions
#const n=2.
r(1..n).
c(1..n).
{queen(X,Y)}:-r(X),c(Y).
#show queen/2.
```

- les commentaires sont précédés d'un `%`
- il est possible de définir des constantes
- il est possible de limiter les atomes à afficher dans les *answer set*

résultat

```
clingo version 5.7.1
Reading from prg.lp
Solving...
Answer: 1

Answer: 2
queen(2,1)
Answer: 3
queen(1,2)
Answer: 4
queen(2,1) queen(1,2)
Answer: 5
queen(2,2)
Answer: 6
queen(1,2) queen(2,2)
Answer: 7
...
Answer: 16
queen(1,1) queen(2,1) queen(1,2) queen(2,2)
SATISFIABLE

Models       : 16
Calls        : 1
Time         : 0.001s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time     : 0.001s
```

utilisation de python

```
from clingo.control import Control

if __name__ == "__main__":
  ctl = Control()
  ctl.configuration.solve.models = 0
  ctl.load("prg.lp")
  ctl.ground()
  ctl.solve(on_model=lambda m: print("Answer: {}".format(m)))
```

- pour une sortie formatée (avec ici, l'ensemble vide en première réponse)

```
$> python prg.py
Answer:
Answer: queen(2,1)
Answer: queen(1,2)
Answer: queen(2,1) queen(1,2)
Answer: queen(2,2)
Answer: queen(1,2) queen(2,2)
Answer: queen(2,1) queen(2,2)
Answer: queen(2,1) queen(1,2) queen(2,2)
Answer: queen(1,1)
Answer: queen(1,1) queen(2,1)
Answer: queen(1,1) queen(2,2)
Answer: queen(1,1) queen(2,1) queen(2,2)
Answer: queen(1,1) queen(1,2)
Answer: queen(1,1) queen(1,2) queen(2,2)
Answer: queen(1,1) queen(2,1) queen(1,2)
Answer: queen(1,1) queen(2,1) queen(1,2) queen(2,2)
```

représentation des entiers

```
#const n=64.
fac(1,1).
fac(N,R*N):-N<n,fac(N-1,R).
true_fac(1,1).
true_fac(N,Res):-N<n,true_fac(N-1,R),Res=R*N,N=Res/R.
```

- vérification par calcul inverse
- des valeurs entières sur 32 bits

```
fac(1,1) fac(2,2) fac(3,6) fac(4,24) fac(5,120) fac(6,720) fac(7,5040) fac(8,40320)
fac(9,362880) fac(10,3628800) fac(11,39916800) fac(12,479001600) fac(13,1932053504)
fac(14,1278945280) fac(15,2004310016) fac(16,2004189184) fac(17,-288522240)
fac(18,-898433024) fac(19,109641728) fac(20,-2102132736) fac(21,-1195114496)
fac(22,-522715136) fac(23,862453760) fac(24,-775946240) fac(25,2076180480)
fac(26,-1853882368) fac(27,1484783616) fac(28,-1375731712) fac(29,-1241513984)
fac(30,1409286144) fac(31,738197504) fac(32,-2147483648) fac(33,-2147483648)
fac(34,0) fac(35,0) fac(36,0) fac(37,0) fac(38,0) fac(39,0) fac(40,0) fac(41,0)
fac(42,0) fac(43,0) fac(44,0) fac(45,0) fac(46,0) fac(47,0) fac(48,0) fac(49,0)
fac(50,0) fac(51,0) fac(52,0) fac(53,0) fac(54,0) fac(55,0) fac(56,0) fac(57,0)
fac(58,0) fac(59,0) fac(60,0) fac(61,0) fac(62,0) fac(63,0)

true_fac(1,1) true_fac(2,2) true_fac(3,6) true_fac(4,24) true_fac(5,120)
true_fac(6,720) true_fac(7,5040) true_fac(8,40320) true_fac(9,362880)
true_fac(10,3628800) true_fac(11,39916800) true_fac(12,479001600)
```

arguments en ligne de commande avec clingo

```
fac(1,1).
fac(N,Res):-N<=n,fac(N-1,R),Res=R*N,N=Res/R.  %% N=Res/R because of int32 limit
res(X):-fac(n,X).
#show.
#show X : res(X).
```

- la constante `n` est définie sur la ligne de commande
- on réduit l'affichage à certaines valeurs dans le programe ASP avec `#show`
- on réduit la sortie de clingo au strict minimum avec `--verbose=0`

```
$> clingo prg.lp -c n=10 --verbose=0
3628800
SATISFIABLE
```

arguments avec python

```
from clingo.control import Control

if __name__ == "__main__":
  if len(sys.argv) != 2:
    print(f"usage: {sys.argv[0]} VAL")
    sys.exit(1)
  ctl = Control()
  ctl.configuration.solve.models = 0
  buf = "#const n=%s." % (sys.argv[1])
  ctl.add("base", [], buf)
  ctl.load("prg.lp")
  ctl.ground()
  ctl.solve(on_model=print)
```

- la constante `n` est définie sur la ligne de commande
- le programme `prg.lp` est chargé à l'exécution

```
$> python prg.py 10
3628800
```

encapsulation complète dans un programme python

```
import sys
from clingo.control import Control

if __name__ == "__main__":
  ctl = Control()
  buf = "#const n=%s." % (10)
  ctl.add("base", [], buf)
  ctl.add("base", [], "number(1..n).")
  ctl.add("base", [], "fac(1,1).")
  ctl.add("base", [], "fac(N,Res):-N<=n,fac(N-1,R),Res=R*N,N=Res/R.")
  ctl.add("base", [], "res(X):-fac(n,X).")
  ctl.add("base", [], "#show.")
  ctl.add("base", [], "#show X : res(X).")

  ctl.ground()
  ctl.solve(on_model=print)
```

- la constante `n` est définie dans le programme python
- les prédicats `number`, `fac` et `res` sont ajoutés un à un le programme python
- les directives d'affichage de clingo sont ajoutées à la fin

```
$> python prg.py
3628800
```

calcul de factoriel

```
#const n=64.
number(1..n).

fac(1,1).
fac(N,Res):-number(N),fac(N-1,R),Res=R*N,N=Res/R.

argmax_fac(X):-X = #max { N : fac(N,_)}.
vmax_fac(X):-argmax_fac(Y),fac(Y,X).

#show argmax_fac/1.
#show vmax_fac/1.
```

- on peut vérifier l'exactitude du calcul par le calcul inverse
- on réduit l'affichage à certains prédicats dans le programe ASP avec `#show`

```
argmax_fac(12) vmax_fac(479001600)
SATISFIABLE
```

calcul de fibonacci

```
#const n=64.
number(1..n).

fib(0, 0).
fib(1, 1).
fib(N, Res) :- number(N), N>1, fib(N-1,X1), fib(N-2,X2), Res=X1+X2, Res>0,
  X1=Res-X2, X2=Res-X1.
max_fib(X):-X = #max { N: fib(N,_)}.
argmax_fib(X,Y):-max_fib(X),fib(X,Y).

fib'(0, 0).
fib'(1, 1).
fib'(N, Res) :- N>1, N<n, fib'(N-1,X1), fib'(N-2,X2), Res=X1+X2, Res>0,
  X1=Res-X2, X2=Res-X1.
max_fib'(X):-X = #max { N: fib'(N,_)}.
argmax_fib'(X,Y):-max_fib'(X),fib'(X,Y).

#show. #show argmax_fib/2. #show argmax_fib'/2.
```

- avec ou sans prédicat `number`.

```
argmax_fib(46,1836311903) argmax_fib'(46,1836311903)
SATISFIABLE
```

le problème des n reines

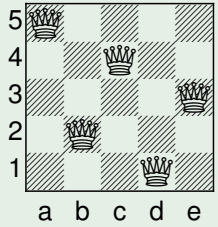
- placer n reines sur une échiquier $n \times n$, sans qu'elles se voient
- générer-tester
- première solution

```
r(1..n).
c(1..n).
{ queen(I, J) } :- r(I), c(J).

:- not n { queen(I,J) } n.
:- queen(I, J), queen(II, J), I!=II.
:- queen(I, J), queen(I, JJ), J!=JJ.
:- queen(I, J), queen(II, JJ), (I, J)!=(II, JJ), I-J==II-JJ.
:- queen(I, J), queen(II, JJ), (I, J)!=(II, JJ), I+J==II-JJ.

#show queen/2.
```

- pour `n=5`.

```
queen(5,1) queen(2,2) queen(4,3) queen(1,4) queen(3,5)
SATISFIABLE
```

Résultat



le problème des n reines (suite)

```
from parse import *
import sys
import time
from clingo.control import Control

START = time.time()

def print_nqueens(m,boardsize):
  global START
  board = "."*boardsize*boardsize
  str_coords = format(m).split()
  for i in range(0, len(str_coords)):
    tmp = parse("queen({},{})",str_coords[i])
    where = int(tmp[1])-1+int((boardsize-int(tmp[0]))*boardsize)
    board = board[:where] + 'Q' + board[where+1:]
  for i in range(0,boardsize):
    board = board[:boardsize*(i+1)+i] + '\n' + board[boardsize*(i+1)+i:]
  print(board)
  now = time.time()-START
  print(round(now,2))

if __name__ == "__main__":
  if len(sys.argv) != 2:
    print(f"usage: {sys.argv[0]} VAL")
    sys.exit(1)
  ctl = Control()
  ctl.configuration.solve.models = 1
  buf = "#const n=%s." % (sys.argv[1])
  ctl.add("base", [], buf)
  ctl.load("asp_000_nqueens.lp")
  ctl.ground()
  ctl.solve(on_model=lambda m: print_nqueens(m,int(sys.argv[1])))
```

Résultat

```
Q....
..Q..
....Q
.Q...
...Q.

0.0
```

optimiser le grounding en exprimant mieux les contraintes

- avoir une règle conditionnelle sur `queen(I, J)` (*i.e.* sol2)
  - remplacer `{ queen(I, J) } :- r(I), c(J).` par `{ queen(I, J) : r(I) } :- c(J).`
- retirer les symétries (*i.e.* sol3)
  - remplacer `J!=JJ` par `J<JJ`
- avoir une règle conditionnelle avec des cardinalité min et max sur `queen(I, J)` (*i.e.* sol4)
  - ajouter des cardinalités inf et sup `1 { queen(I, J) : r(I) } 1 :- c(J).`
- temps en sec pour trouver une solution (sur Apple M1 Max 32Go)

boardsize	sol1	sol2	sol3	sol4
12	<0.1	<0.1	<0.1	<0.1
13	<0.1	<0.1	<0.1	0.1
14	9.4	3.5	3.6	<0.1
15	39.7	38.9	40.3	<0.1
...	-	-	-	...
86	-	-	-	4.4
87	-	-	-	4.7
88	-	-	-	4.9
89	-	-	-	5.2
...	-	-	-	...
199	-	-	-	120

optimiser le grounding en exprimant mieux les contraintes (suite)

- ajouter des contraintes sur les lignes et les colonnes (*i.e.* sol5)
  - les contraintes

```
:- not n { queen(I,J) } n.
:- queen(I, J), queen(II, J), I<II.
:- queen(I, J), queen(I, JJ), J<JJ.
```

  - deviennent

```
:- r(I), not 1 { queen(I, J) } 1.
:- c(J), not 1 { queen(I, J) } 1.
```

boardsize	sol1	sol2	sol3	sol4	sol5
12	<0.1	<0.1	<0.1	<0.1	<0.1
13	<0.1	<0.1	<0.1	<0.1	<0.1
14	9.4	3.5	3.6	<0.1	<0.1
15	39.7	38.9	40.3	<0.1	<0.1
...	-	-	-	...	
86	-	-	-	4.4	3.7
87	-	-	-	4.7	3.8
88	-	-	-	4.9	4.0
89	-	-	-	5.2	4.2
...	-	-	-	...	...
98	-	-	-	7.5	5.8
99	-	-	-	7.8	6.0
100	-	-	-	8.1	6.3
...	-	-	-	...	...
198	-	-	-	117.8	97.0
199	-	-	-	120.0	99.1
200	-	-	-	-	101.4
...	-	-	-	-	...
210	-	-	-	-	119.4

optimiser le grounding en exprimant mieux les contraintes (suite)

- ajouter des contraintes sur les diagonales (*i.e.* sol6)
  - les contraintes

```
:- queen(I, J), queen(II, JJ),
   I<II, I-J==II-JJ.
:- queen(I, J), queen(II, JJ),
   I<II, I+J==II+JJ.
```

  - deviennent

```
:- D = 1..n*2-1,
   not { queen(I,J) : d1(I,J,D) } 1.
:- D = 1..n*2-1,
   not { queen(I,J) : d2(I,J,D) } 1.
d1(I,J,I-J+n):- r(I), c(J).
d2(I,J,I+J-1):- r(I), c(J).
```

boardsize	sol1	sol2	sol3	sol4	sol5	sol6
15	39.7	38.9	40.3	<0.1	<0.1	<0.1
...	-	-	-	...	...	...
199	-	-	-	120.0	99.1	0.3
210	-	-	-	-	119.4	0.4
1000	-	-	-	-	-	13.8
2000	-	-	-	-	-	66.1
2510	-	-	-	-	-	112.5

conclusion

- concepts et algorithmes de PL à ASP
- tests sur le problème des n reines avec 120sec max de calcul
- PL (avec SLDNF)
  - des faits et des règles
  - absence possible de solutions selon l'ordre des déclarations
  - **27 reines** et des temps de calcul très variable
- CLP
  - des domaines pour restreindre les affectations
  - 29 reines sans politique et **87 reines** avec
- SAT (avec DPLL-CDCL)
  - parcours en fonction des clauses en apprenant à partir des erreurs
- SMT (SAT avec des domaines)
- ASP (avec CDNL)
  - parcours en fonction des nogoods
  - **2510 reines** sans politique

perspectives

- s'amuser à résoudre d'autres puzzles
- tester d'autres solver asp (ASPeRIX, DLV, nomore++, Platypus)
- tester d'autres *grounder* que gringo (I-DLV, CASP, ASP-MT, HEX, ProASP)
- tester d'autres *solver* que clasp (Cmodels, diff-SAT, GnT, Smodels)
- tester les politiques possibles dans clasp
  - préprocesser en itératif/distribué/parallèle
  - si plusieurs solutions sont demandées, mémoriser les nogoods correspondant à une solution (`enum-mode`)
  - si un optimum est demandé, mémoriser certains nogoods et réaliser des *restart* (`restart-mode`)
  - utiliser les heuristiques de parcours de SAT (`viz, bermin, vsids, vmtf`)
  - utiliser les heuristiques de restart (arimthétique, géométrique, *luby*, *nested*)
  - utiliser les heuristiques de supression des nogood (`dfrac, dglue`)
  - paralléliser avec des nogoods partagés et des nogoods distribués (`clasp2`)

bibliographie

- *Answer Set Solving in Pratice*, M. Gebser, R. Kaminski, B. Kaufmann, and T. Schaub
  Synthesis Lectures on Artificial Intelligence and Machine Learning, Morgan and Claypool (2012)
- *Potsdam Answer Set Solving Collection* (University of Potsdam, 2003-2026)
  Online at `https://potassco.org/`
- *The Art of Prolog: Advanced Programming Techniques*, L. Sterling
  Mit Press Series in Logic Programming (1986)
- *The Power of Prolog*, M. Triska (2005-2026)
  Online at `https://www.metalevel.at/prolog`
- *SAT/SMT by Example*, D. Yurichev (2015-2026)
  Online at `https://smt.st/`
- *Investigating the Grounding Bottleneck for a Large-Scale Configuration Problem: Existing Tools and Constraint-Aware Guessing*, V. Semmelrock and G. Friedrich, 41nd International Conference on Logic Programming (ICLP-2025)
- *19th Workshop on Answer Set Programming and Other Computing Paradigms* (ASPOCP 2026, July 18, Lisbon, Portugal), at ICLP-2026